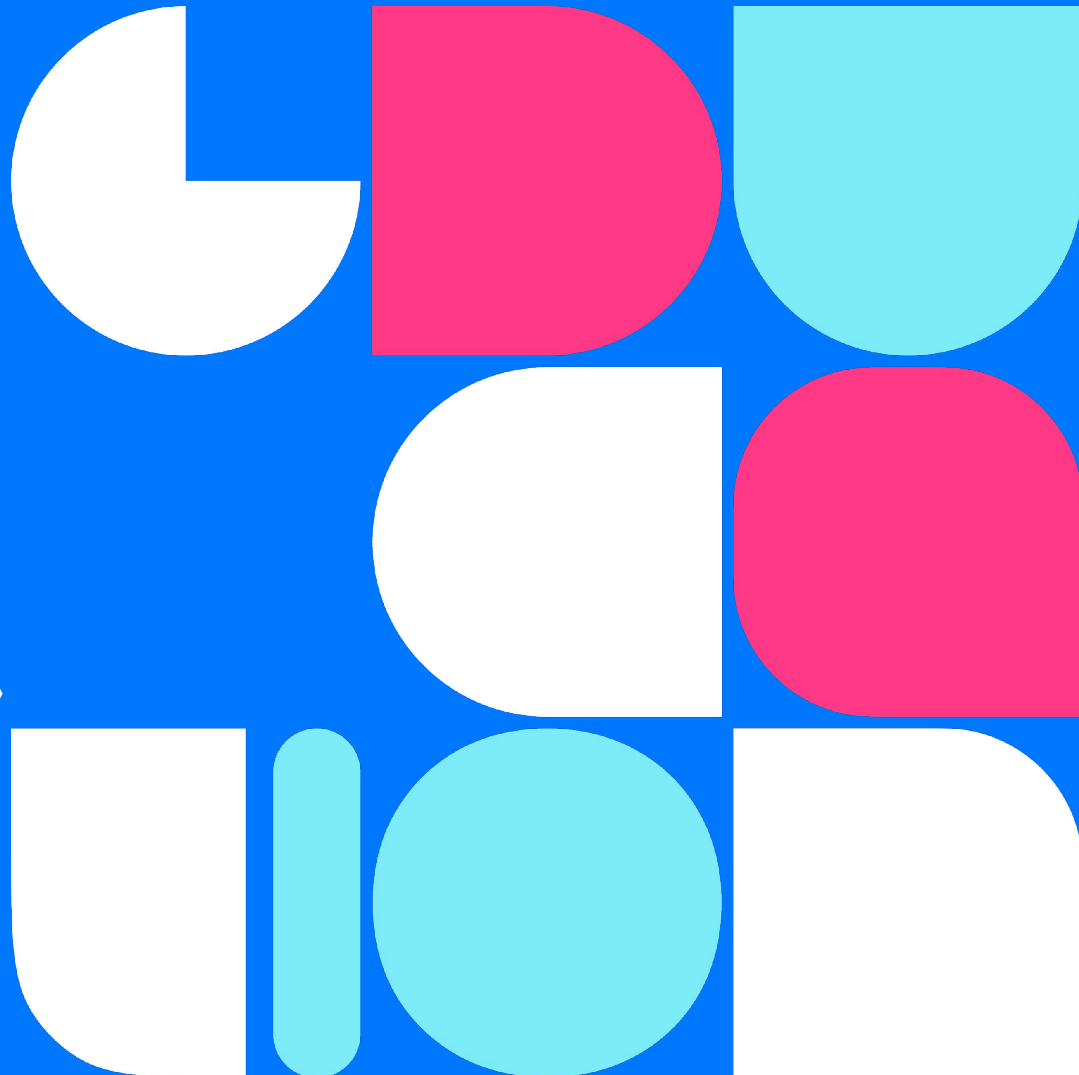
education

Предобработка запросов. Исправление опечаток

«Информационный поиск и ранжирование»
Преподаватель Руслан Аллаяров



Ведущий лекции



- Более 7 лет в ML, 6 лет из них в поиске
- Lead developer (реком товаров Ozon), Ex VK (поиск), Ex ОККО (реком главной витрины)
- Отвечал за блендинг поиска по видео, за базовое качество поиска по клипам и плейлистам
- Закончил мехмат МГУ и Техносферу (2021)

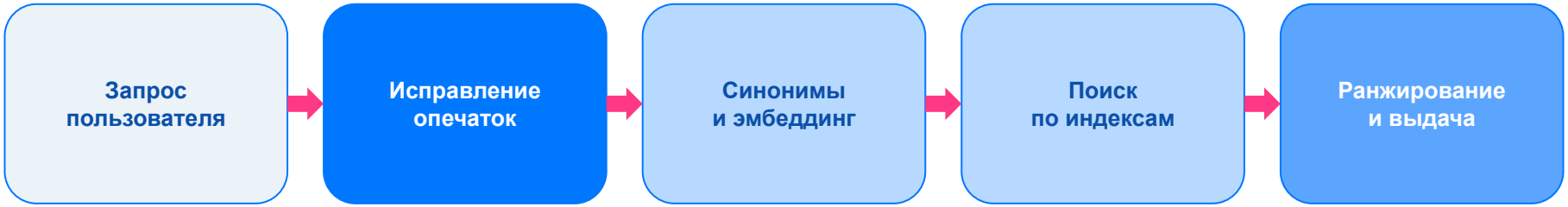
education

План лекции

1. Опечатки в поисковых запросах
2. Простой опечаточник
3. Близость слов: Левенштейн
4. Модель зашумлённого канала
5. Генерация кандидатов
6. Современные методы
7. Итоги и материалы
8. Домашнее задание

Где мы находимся

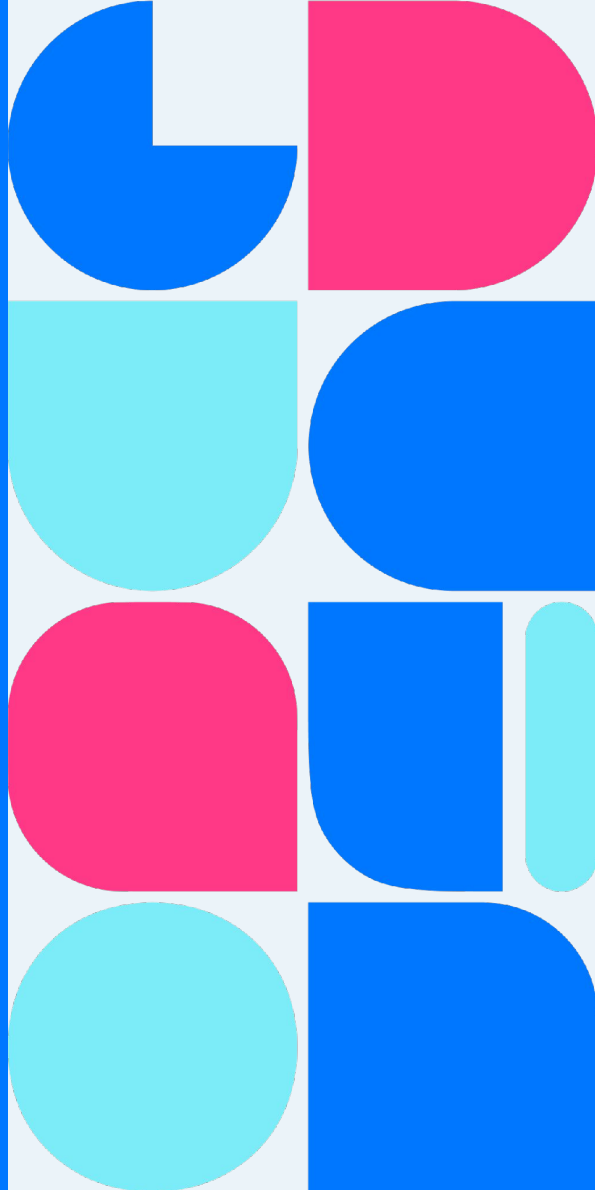
- В лекции 1 мы разобрали онлайн-архитектуру: запрос сначала проходит предобработку, и только потом идёт в индексы
- Предобработка — это исправление опечаток, расширение синонимами и эмбеддинг запроса
- Сегодня разбираем первый и самый заметный для пользователя блок — исправление опечаток



сегодняшняя лекция — про второй блок

Опечатки в запросах

Понять, какие бывают опечатки в поисковых запросах и чем они отличаются от опечаток в обычном тексте



Виды опечаток в запросах

По разным оценкам 10–20 % поисковых запросов содержат хотя бы одну опечатку. Для поиска это прямая потеря трафика: без исправления запрос просто не находит ничего.

Тип	Ввели	Имели в виду
Замена буквы	подслушено	подслушано
Пропуск буквы	послушано	подслушано
Лишняя буква	подслушанно	подслушано
Перестановка	Александр	Александр
Пропуск пробела	ленинградскийвокзал	ленинградский вокзал
Лишний пробел	вело сипед	велосипед
Раскладка	ghbdtn	привет
Транслит	privet	привет

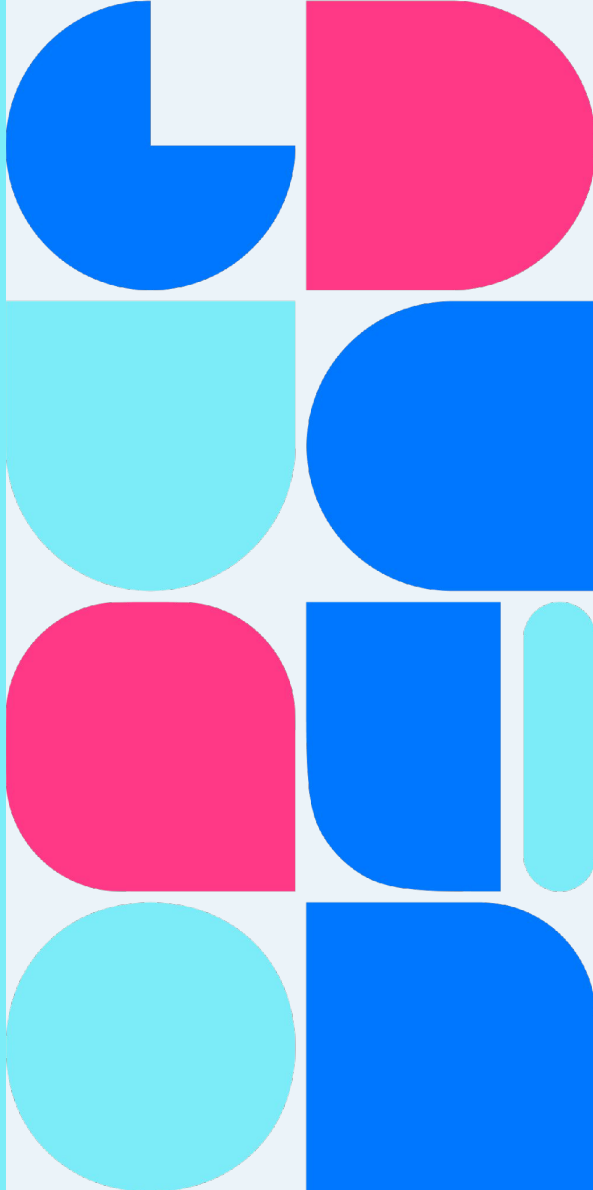
Поиск против текстового редактора

Это важно: спеллчекер из текстового редактора в поиске работает плохо, и вот почему.

Нет контекста	Запрос — это 2–3 слова, а не абзац. Языковой модели почти не на что опереться
Много имён собственных	«дайвинчик», «вкусвилл», названия игр и брендов — их нет ни в одном орфографическом словаре
Словарь меняется каждый день	Появился новый сериал — назавтра его пишут с ошибками миллион раз
Цена ошибки несимметрична	Не исправили — пользователь увидит пустую выдачу. Исправили неверно — увидит чужую выдачу и уйдёт
Жёсткий бюджет времени	Исправление — часть обработки запроса, на всё про всё десятки миллисекунд

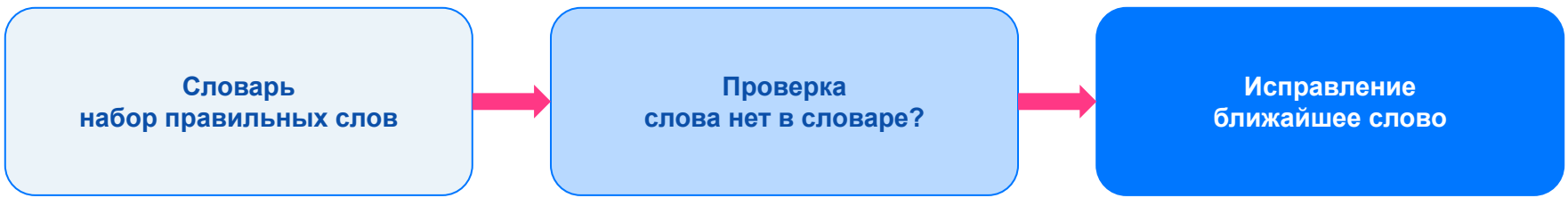
Простой опечаточник

Собрать самый наивный исправитель на словаре и увидеть, на чём именно он ломается



Простой опечаточник

- Составляем словарь правильных слов
- Если слово запроса в словаре не нашлось — считаем, что это опечатка
- В качестве исправления предлагаем ближайшее слово из словаря
- Так устроены спеллчекеры текстовых редакторов: GNU Aspell, Hunspell (Chrome, Firefox, LibreOffice, Telegram)



два вопроса без ответа: откуда взять словарь и что значит «ближайшее»

Откуда взять словарь

Два источника, и у обоих есть своя беда.

Орфографический словарь

1. Чистый: почти нет мусора
2. Низкая полнота: нет имён, брендов, сленга, новых слов
3. Обновляется раз в несколько лет

Корпус текстов и логи запросов

1. Высокая полнота: есть всё, что реально ищут
2. Есть мусор: чужие опечатки попадают в словарь как «правильные»
3. Лечится порогом по частоте и фильтрами

На практике берут второе и чистят: слово попадает в словарь, если встретилось достаточно часто у достаточного числа пользователей

Где простой опечаточник ломается

Проблема 1: кандидатов много, и они равноценны

запрос «татья» — расстояние 1 сразу до десятка слов:

статья	татьян	третья	татьяна	статьи	таиться	татьяне	статью	тетя	катя

по расстоянию выбрать нельзя — нужен ещё какой-то критерий

Проблема 2: словарь

«Давинчик» → «Дайвинчик»

Правильное слово — название приложения. Его нет в орфографическом словаре, а в логах оно встречается в обоих написаниях.

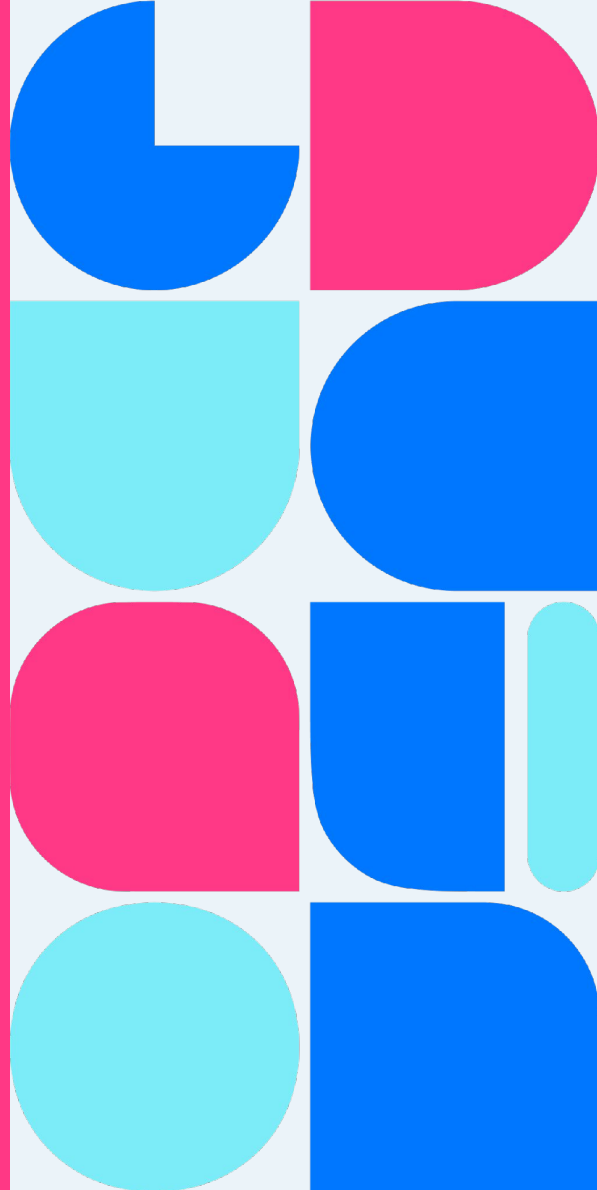
Проблема 3: нет контекста

«наталная карта» → «натальная карта»

Слово «наталная» само по себе можно исправить десятком способов. Правильный выбор подсказывает только соседнее слово «карта».

Близость слов

Научиться измерять расстояние между словами и строить редакционное предписание



Расстояние Левенштейна

- Минимальное число операций, которыми одно слово превращается в другое
- Базовые операции: замена символа, вставка символа, удаление символа
- Если добавить транспозицию (перестановку двух соседних символов) — получится расстояние Дамерау-Левенштейна
- Транспозиция важна: это самая частая опечатка при быстром наборе («Александр»)

замена	вставка	удаление	транспозиция
кот → кит	кот → крот	крот → кот	кот → окт
замена о → и	вставили р	удалили р	поменяли к и о

Дамерау показал, что около 80 % опечаток — это ровно одна из этих четырёх операций

Алгоритм Вагнера — Фишера

- Расстояние считается динамическим программированием по таблице (M+1) × (N+1)
- D[i][j] — расстояние между первыми i буквами первого слова и первыми j буквами второго
- Первая строка и первый столбец заполняются числами 0…N и 0…M: это стоимость превратить слово в пустую строку

Рекуррентное соотношение для остальных ячеек

`D[i][j] = min из трёх вариантов:`

`D[i-1][j] + 1`	удаление буквы
`D[i][j-1] + 1`	вставка буквы
`D[i-1][j-1] + c`	замена; c = 0 если буквы совпали, иначе 1

Шаг 0: инициализация

Заполняем нулевую строку и нулевой столбец. Остальное пока неизвестно.

		с	а	р	а	т	о	в
	0	1	2	3	4	5	6	7
с	1							
а	2							
м	3							
а	4							
р	5							
а	6							

Что означают эти числа

Нулевая строка — во что обходится превратить пустую строку в префикс слова «саратов». Чтобы получить «сар», надо вставить три буквы, значит 3.

Нулевой столбец — наоборот: сколько удалений нужно, чтобы от префикса «сам» дойти до пустой строки. Тоже 3.

Дальше каждая клетка считается только через трёх соседей: сверху, слева и по диагонали.

Шаг 1: первая строка

Строка для буквы «с». Разберём клетку (с, с) — первую настоящую.

		с	а	р	а	т	о	в
	0	**1**	2	3	4	5	6	7
с	**1**	**0**	1	2	3	4	5	6
а	2							
м	3							
а	4							
р	5							
а	6							

Клетка (с, с)

Три кандидата:

сверху D[0][1] + 1 = 2
слева D[1][0] + 1 = 2
диагональ D[0][0] + 0 = 0

Буквы совпали, поэтому по диагонали доплачивать не нужно. Минимум = 0.

Голубым отмечены три соседа, розовым — считаемая клетка.

Шаг 2: когда буквы не совпали

Строка «м», клетка (м, р): буквы разные.

		с	а	р	а	т	о	в
	0	1	2	3	4	5	6	7
с	1	0	1	2	3	4	5	6
а	2	1	**0**	**1**	2	3	4	5
м	3	2	**1**	**1**	2	3	4	5
а	4							
р	5							
а	6							

Клетка (м, р)

Три кандидата:

сверху D[2][3] + 1 = 2
слева D[3][2] + 1 = 2
диагональ D[2][2] + 1 = 1

Буквы «м» и «р» разные, поэтому к диагонали прибавляем 1 — это замена.

Минимум всё равно даёт диагональ: одна замена дешевле, чем удаление плюс вставка.

Как мы пришли в угол

Подсвечен путь, по которому набралось это число 4.

		с	а	р	а	т	о	в
	0	1	2	3	4	5	6	7
с	1	**0**	1	2	3	4	5	6
а	2	1	**0**	1	2	3	4	5
м	3	2	1	**1**	2	3	4	5
а	4	3	2	2	**1**	**2**	3	4
р	5	4	3	2	2	2	**3**	4
а	6	5	4	3	2	3	3	**4**

путь восстанавливается обратным ходом — из правого нижнего угла

Что говорит путь

с	совпадение
а	совпадение
м → р	**замена**
а	совпадение
+ т	**вставка**
р → о	**замена**
а → в	**замена**

Три замены и одна вставка — ровно 4, как и в углу матрицы.

Редакционное предписание

- Матрица даёт не только число, но и сам список операций
- Идём из правого нижнего угла в левый верхний, каждый раз выбирая ту клетку, из которой пришло минимальное значение
- По направлению шага восстанавливаем операцию

Как читать направление

↘	**по диагонали**	замена или совпадение
→	**влево**	вставка буквы (из второго слова)
↓	**вверх**	удаление буквы (из первого слова)

Зачем это нужно дальше

Редакционное предписание — это и есть список конкретных ошибок, которые сделал пользователь.

Именно по нему в модели ошибок будут считаться вероятности: какие буквы люди путают чаще.

Тем же приёмом SAGE собирает статистику для генерации синтетических опечаток.

Сложность и что с ней делают

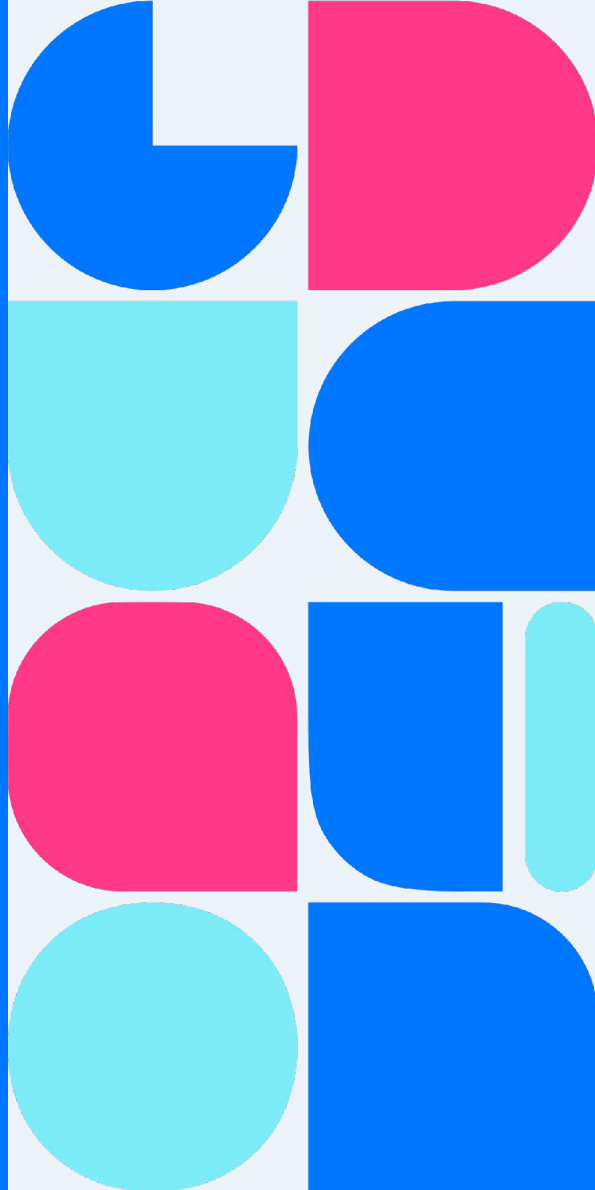
- Заполнение матрицы — $O(M \cdot N)$ по времени и по памяти
- Память легко снижается до $O(\min(M, N))$: при построчном проходе нужны только две соседние строки
- Если нас интересует только «расстояние не больше k», считаем лишь диагональную полосу шириной $2k+1$ — получается $O(k \cdot N)$
- Для поиска по словарю это всё равно слишком дорого: считать расстояние до каждого из миллиона слов нельзя
- Поэтому кандидатов сначала отбирают структурой данных, а расстояние считают уже для десятков слов, а не для миллиона

Запомнить

Левенштейн — это способ оценить пару слов, а не способ искать по словарю. Поиск — отдельная задача, к ней вернёмся в разделе про генерацию кандидатов.

Модель зашумлённого канала

Перейти от «просто ближайшего слова» к вероятностной постановке задачи



Зашумлённый канал: постановка

- Метафора: пользователь «передаёт» нам правильный запрос по зашумлённому каналу, и канал портит символы
- Наша задача — по искажённому запросу восстановить исходный
- Обозначения: *Q_typed* — то, что ввели; *Q_fix* — то, что имели в виду; *D* — множество всех возможных запросов

Формально мы ищем

$$Q_fix = \operatorname{argmax} P(Q_fix \mid Q_typed), \quad Q_fix \in D$$

Прямо посчитать P(Q_fix | Q_typed) нельзя: у нас нет статистики «что имел в виду пользователь» для всех пар запросов. Поэтому выражение разворачивают по формуле Байеса — следующий слайд.

Разворачиваем по Байесу

$$P(Q_fix \mid Q_typed) = P(Q_typed \mid Q_fix) \cdot P(Q_fix) \;/\; P(Q_typed)$$

Знаменатель P(Q_typed) одинаков для всех кандидатов — на argmax он не влияет, его выбрасываем. Остаётся произведение двух множителей.

P(Q_typed | Q_fix)

Модель ошибок

Насколько правдоподобно, что человек, желая набрать Q_fix, набрал именно Q_typed.

Отвечает за вопрос «похоже ли это на опечатку».

P(Q_fix)

Языковая модель

Насколько вообще вероятен запрос Q_fix сам по себе.

Отвечает за вопрос «так вообще говорят».

Именно второй множитель решает проблему «наталная карта»: контекст приходит отсюда

Модель ошибок: что это такое

- Вероятность конкретного искажения раскладывается на вероятности отдельных операций
- P(«собраный» | «собранный») ≈ P(пропуск «н» после «н»)
- P(«соьранный» | «собранный») ≈ P(замена «б» → «ь»)
- То есть нам нужна таблица: для каждой пары букв — насколько часто одну путают с другой
- Это называется матрицей ошибок (confusion matrix): отдельные таблицы для замен, вставок, удалений и транспозиций

Почему это лучше простого расстояния

Расстояние Левенштейна считает все ошибки одинаковыми. А на клавиатуре «б» и «ь» — соседи, и путают их в сотни раз чаще, чем «б» и «ю». Модель ошибок это знает, обычное расстояние — нет.

Откуда взять статистику ошибок

Нужен параллельный корпус: пары «как ввели → как правильно». Дальше по каждой паре строим редакционное предписание и считаем, какие операции встречаются чаще.

Процедура

1. Строим матрицу Левенштейна для пары
2. Идём backtrace и получаем список операций
3. Инкрементируем счётчики в матрице ошибок
4. Нормируем в вероятности

Готовые датасеты

KartaSlov (kartaslov.ru) — открытый русскоязычный ресурс с парами опечаток

RUSpellRU, MultidomainGold — размеченные корпуса из бенчмарка SAGE

Поведенческая статистика

Пользователь ввёл запрос, ничего не кликнул, тут же переформулировал и кликнул — это почти наверняка пара «опечатка → исправление»

Самый ценный источник: ошибки именно ваших пользователей

Взвешенное расстояние

- Модель ошибок встраивается прямо в алгоритм Вагнера — Фишера
- Вместо «+1» за каждую операцию подставляем её стоимость: −log вероятности этой операции
- Частая ошибка стоит дёшево, редкая — дорого. Минимизация суммы стоимостей = максимизация вероятности искажения
- Логарифмы нужны, чтобы произведение вероятностей превратилось в сумму — с ней умеет работать динамическое программирование

Как меняется рекуррентное соотношение

`D[i−1][j] + w_del(a)`	было: + 1
`D[i][j−1] + w_ins(b)`	было: + 1
`D[i−1][j−1] + w_sub(a,b)`	было: + 0 или + 1

Языковая модель

- Самый простой вариант: P(запроса) = частота этого запроса в логах, делённая на общее число запросов
- Но длинные и редкие запросы в логах встречаются ноль или единицы раз — а именно их и приходится исправлять: частотные запросы люди набирают правильно
- Ноль — это не «маленькая вероятность», а отсутствие сигнала: двух кандидатов с нулём модель различить не может
- Разбиваем запрос на слова и делаем предположение о независимости: P(q) = произведение P(слова)
- Но тогда мы снова теряем контекст — а именно за ним мы сюда и шли

Компромисс: цепь Маркова и биграммная модель

$$P(w_1 \dots w_n) \approx \prod P(w_i \mid w_{i-1})$$

Вероятность слова зависит только от предыдущего. Для запроса из 2–3 слов этого достаточно, а статистику по парам слов из логов собрать уже реально. Именно здесь «карта» вытягивает «натальную».

Сглаживание

- Проблема: если пара слов в логах не встретилась ни разу, её вероятность равна нулю — и весь запрос получает нулевую вероятность
- Один невиданный биграм обнуляет кандидата целиком, даже если всё остальное в нём отлично
- Сглаживание перераспределяет немного вероятностной массы с виденных событий на невиданные

Аддитивное (Лапласа)	Прибавляем к каждому счётчику α. Просто, но грубо: раздаёт массу всем подряд одинаково
Backoff	Не видели биграм — откатываемся к униграмной оценке со штрафом
Интерполяция	Всегда смешиваем биграм и униграм с весами: $\lambda \cdot P(w|w_{-1}) + (1-\lambda) \cdot P(w)$
Кнесера — Нея	Учитывает не только частоту слова, но и разнообразие контекстов, в которых оно встречалось. Стандарт для n-грамм

Собираем скор кандидата

Финальная оценка кандидата — сумма двух логарифмов. Обе части мы уже умеем считать.

$$\text{score}(Q_fix) = \log P(Q_typed \mid Q_fix) + \lambda \cdot \log P(Q_fix)$$

Первое слагаемое

Взвешенное расстояние Дамерау-Левенштейна между введённым словом и кандидатом.
Чем правдоподобнее опечатка — тем выше.

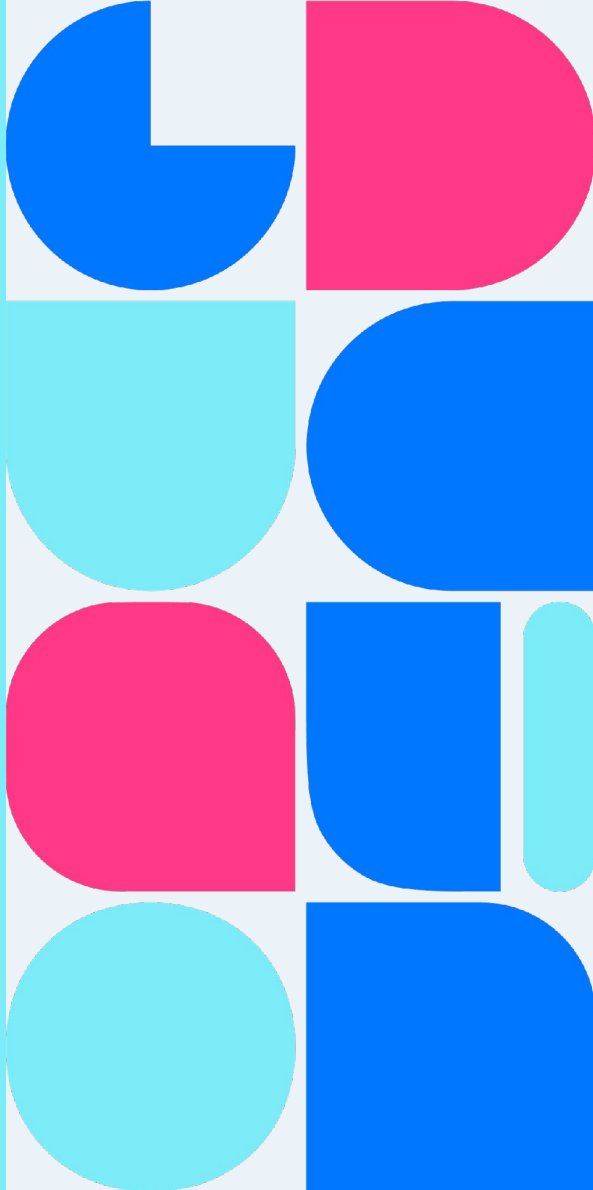
Второе слагаемое

Биграммная языковая модель со сглаживанием.
Чем естественнее запрос — тем выше.

λ подбирается на отложенной выборке. Он управляет главным компромиссом: маленькая λ — доверяем написанию и правим осторожно, большая — доверяем популярности и правим агрессивно.

Генерация кандидатов

Понять, как искать кандидатов быстро, не перебирая весь словарь



Кандидаты: перебор наизнанку

Оценивать пару «слово — кандидат» мы умеем. Но кандидатов надо сначала откуда-то взять.

В лоб

Посчитать расстояние до каждого слова словаря.

Миллион запусков динамического программирования, каждый O(n · m). В бюджет запроса не влезает.

Наизнанку

Породить все строки на расстоянии 1 от запроса и проверить каждую в словаре хэш-лукапом.

Расстояние вообще не считается — оно зашито в способ порождения.

Что порождаем для слова «кот»

удаления	от, кт, ко	3 штуки
транспозиции	окт, кто	n − 1
замены	аот, бот, … коа, коб, …	33n
вставки	акот, … кота, котб, …	33(n + 1)

Почти всё — мусор вроде «щот». Оставляем те строки, которые нашлись в словаре: «рот», «кит», «код», «крот»

Порог k и цена расстояния 2

Метод дешёвый, но ровно до k = 1. Дальше он взрывается.

Сколько строк порождается

$$n + (n - 1) + 33n + 33(n + 1) = 68n + 32$$

Слово из 6 букв → около 440 строк. Это 440 хэш-лукапов, доли миллисекунды.
Для k = 2 каждую из 440 надо прогнать ещё раз: примерно 200 тысяч строк. Уже не годится.

Почему порог вообще ставят

Около 80% опечаток — ровно одна правка, k ≤ 2 покрывает почти весь остаток.

На практике порог зависит от длины слова: 1 для коротких, 2 для средних, 3 для длинных.

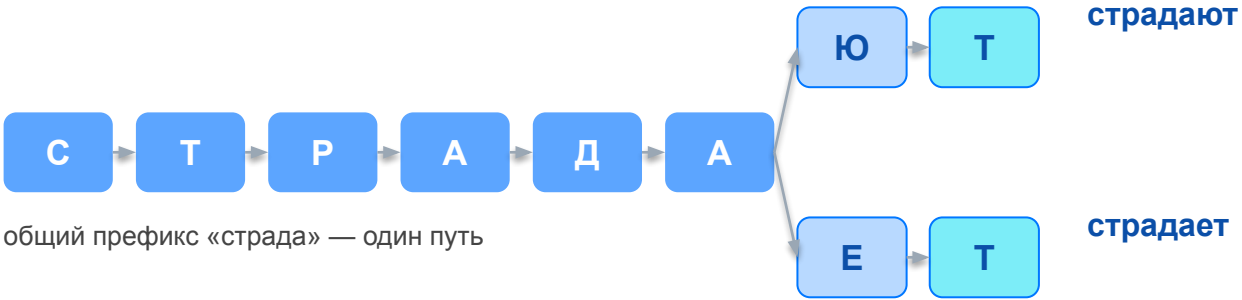
Всё, что дальше порога, метод не видит — и это честное ограничение, а не баг.

Что дальше

Порог k есть у любого метода этого класса — и у trie с отсечкой, и у SymSpell, и у BK-дерева. Разница только в цене перебора при одном и том же k.
SymSpell — прямое развитие этой же идеи: генерировать только удаления, чтобы из формулы ушёл множитель 33.

Префиксное дерево

Узел — буква. Путь от корня — префикс. Поддерево — все слова словаря с этим префиксом.



общий префикс «страда» — один путь

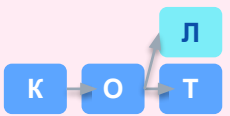
Что хранится в узле

- Ссылки на детей по буквам
- Флаг «здесь кончается слово словаря» — на схеме бирюзовые узлы
- При желании — частота слова, чтобы сразу ранжировать кандидатов

Как строится: офлайн, один раз

1. Берём словарь: правильные слова и их частоты
2. Вставляем слово за словом от корня, нет ребёнка — создаём
3. В последнем узле ставим флаг конца слова

«кот», затем «кол»



путь «ко» переиспользован

Одна строка матрицы на весь узел

Пришёл запрос «страдют». Столбцы матрицы — это запрос, строки — спуск по дереву.

		с	т	р	а	д	ю	т
	0	1	2	3	4	5	6	7
С	1	0	1	2	3	4	5	6
Т	2	1	0	1	2	3	4	5
Р	3	2	1	0	1	2	3	4
А	4	3	2	1	0	1	2	3
Д	5	4	3	2	1	0	1	2
А	6	5	4	3	2	1	1	2
Ю	7	6	5	4	3	2	1	2
Т	8	7	6	5	4	3	2	1
Е	7	6	5	4	3	2	2	2
Т	8	7	6	5	4	3	3	2

Строки 1–6 — путь «страда». Считаются ровно один раз.

ветка «…ют» дописала 2 строки

ветка «…ет» взяла строку 6 и дописала свои

Какое расстояние тут считается

Обычное, невзвешенное: все операции по 1.

Оно целочисленное и монотонное — только поэтому на следующем слайде сработает отсечка.

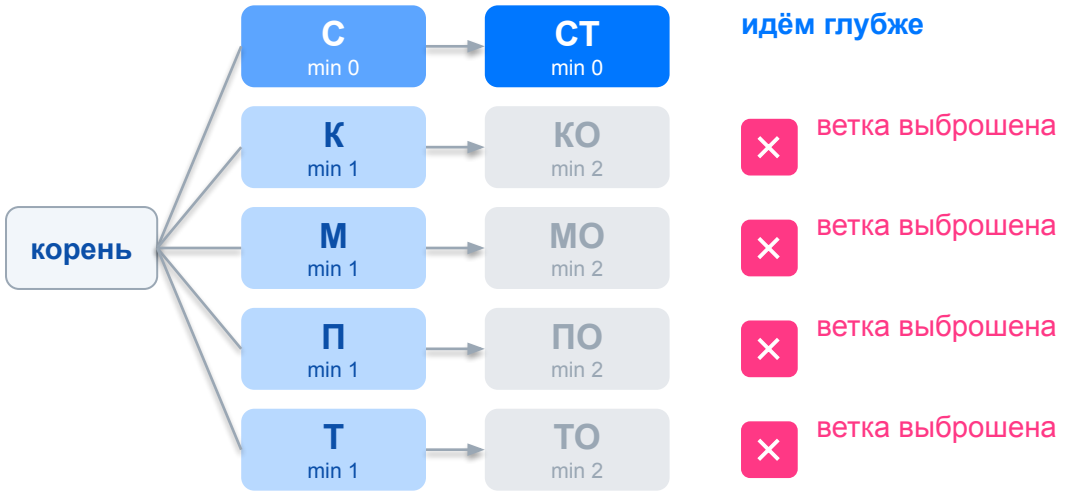
Взвешенное расстояние и языковая модель — это уже ранжирование, на десятке отобранных кандидатов.

Что это даёт

1. Переиспользование строк — выигрыш в разы: узлов в дереве меньше, чем суммарная длина всех слов
2. Главная экономия на следующем слайде: по минимуму в строке можно обрезать целую ветку — это уже тысячи раз
3. И второе возможно только благодаря первому: строка поддерева посчитана до того, как мы в него зашли

Обход дерева с отсечкой

Тот же запрос «страдают», порог k = 1. Решение принимается по минимуму в строке.



строка для «ко»:

2	2	2	3	4	5	6	7

минимум 2 > k = 1 → ни одно слово на «ко…» в порог не уложится

Почему это корректно

Дописывая буквы к кандидату, расстояние нельзя опустить ниже текущего минимума по строке.

Значит минимум — честная оценка снизу для всего поддерева, и превысив k, он уже не вернётся.

Масштаб

При k = 1 почти все ветки умирают на втором-третьем уровне: из миллиона слов доходят тысячи.

Решаем по той самой строке, что считали на прошлом слайде — одна строка на узел

Что происходит на запрос

Соберём три предыдущих слайда в один цикл. Дерево уже построено и от запроса не зависит.

Онлайн: на каждое слово запроса

1. Заводим нулевую строку матрицы: [0, 1, …, |q|] — по длине запроса
2. Идём в глубину от корня. Спускаясь на букву, дописываем к матрице одну новую строку
3. min(строка) > k → возврат: всё поддерево отсекается
4. Узел терминальный и последнее число строки ≤ k → слово попадает в кандидаты
5. Возвращаясь вверх, строку выбрасываем

Матрица нигде не хранится

Её столбцы — это запрос, поэтому она своя на каждый запрос и живёт только по ходу обхода.

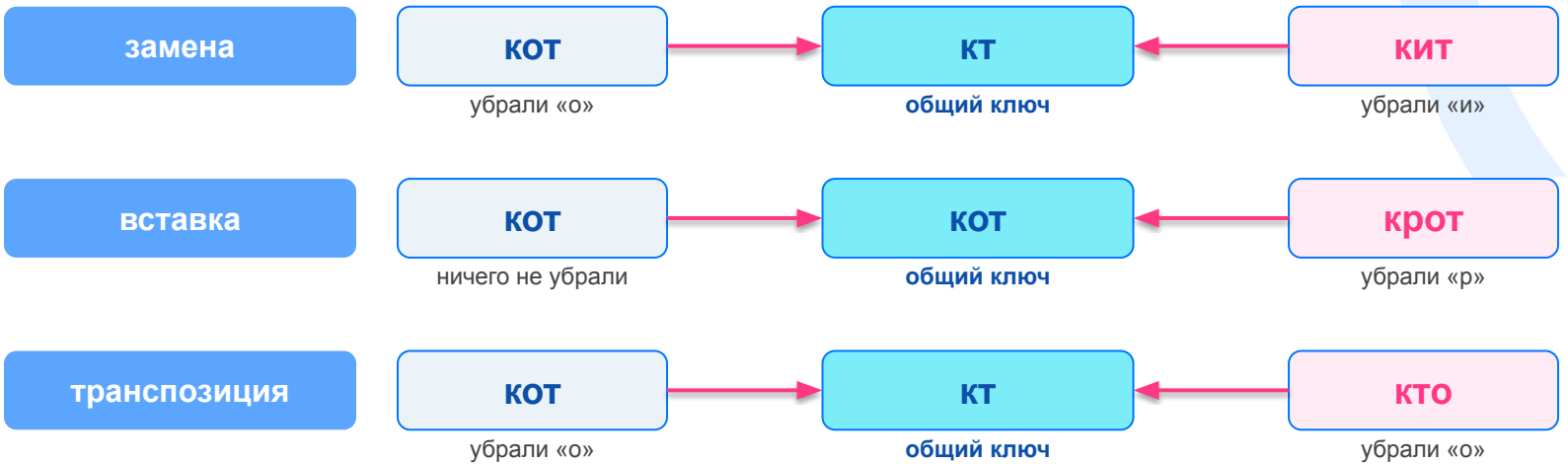
Сколько это памяти

Стек строк глубиной с текущий спуск. Для слова из 8 букв — несколько десятков чисел.

На выходе — десятки кандидатов. Дальше их ранжирует модель зашумлённого канала

SymSpell: почему хватает удалений

Порождать замены и вставки дорого — они множатся на размер алфавита. Оказывается, достаточно одних удалений: любая правка превращается в пару удалений с двух сторон, и слова встречаются на общем ключе.



Что это меняет

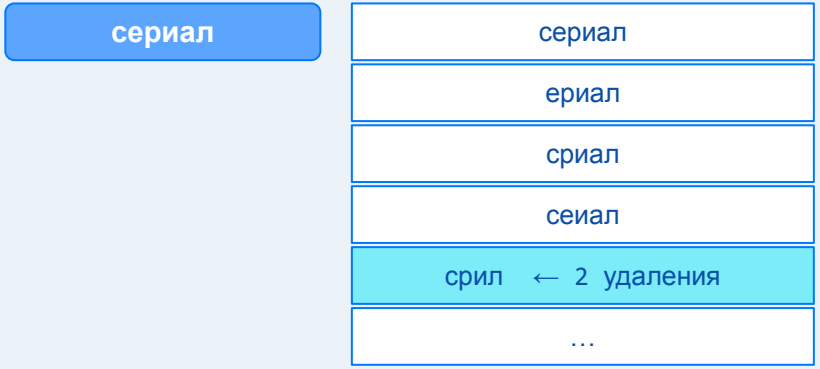
Из формулы 68n + 32 уходит множитель 33: удалений всего n штук на слово, а не 33n замен и 33(n+1) вставок.

SymSpell: офлайн и онлайн

Индекс — хэш-таблица «ключ → слова словаря». Ключи получены удалениями до k букв.

Офлайн: строим индекс

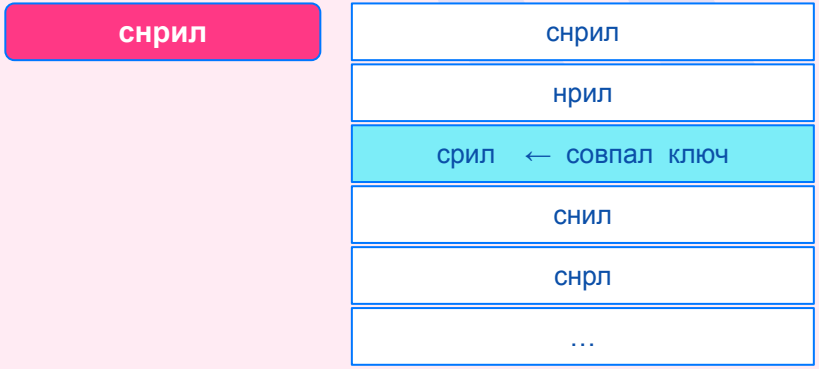
для каждого слова — все удаления до k



всё это — ключи, значение = «сериал»

Онлайн: обрабатываем запрос

то же самое делаем с введённым словом



ищем каждый ключ в хэш-таблице

Чем платим

Размер индекса: при k = 2 словарь раздувается в десятки раз. Зато поиск — это несколько хэш-лукапов по O(1), без обхода дерева и без вычисления расстояния.

*Фонетические индексы

Ещё один способ сузить перебор: присвоить словам код, одинаковый для похоже звучащих слов.

Как считается Soundex

1. Первая буква остаётся как есть
2. Гласные, «h», «w», «y» выбрасываются
3. Согласные заменяются цифрами по группам близких звуков
4. Код обрезается до 4 символов

Канонический пример

`Robert` → `Rupert` → **R163**

Что это даёт: бакеты

слова с одним кодом лежат вместе — перебираем только бакет

код A
- Шмидт
- Шмит

код B
- Фёдоров
- Фёдаров

для русского берут адаптации Soundex и Metaphone

Где работает, а где нет

\+ Ошибки «на слух»: человек не знает, как пишется слово

– Ошибки «от клавиатуры»: сосед по клавише звучит иначе — а в поиске таких большинство

BK-дерево: как строится

Строится офлайн, один раз из словаря — как и префиксное дерево. В онлайне по нему только ищут.

Правило вставки

У узла не может быть двух детей с одинаковой меткой

1. Первое слово словаря становится корнем
2. Для w стоим в узле v, считаем d = d(w, v)
3. Нет у v ребёнка с меткой d — вешаем w туда
4. Есть — спускаемся в него и повторяем

Коллизия по метке проталкивает слово глубже — из-за неё дерево и ветвится.

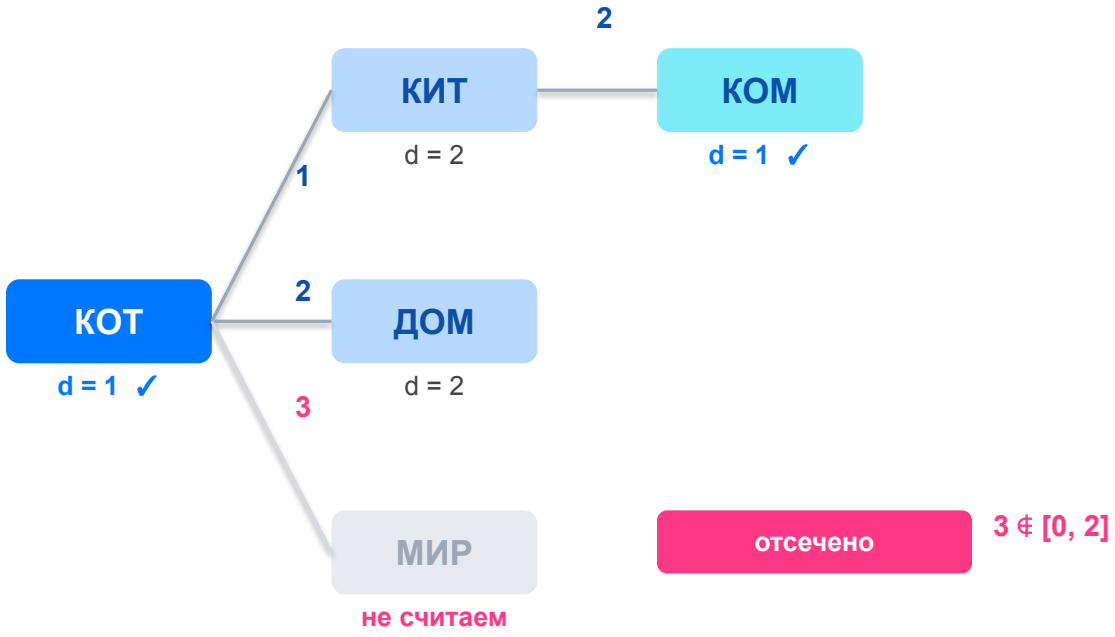
Собираем дерево из словаря

- **кот** — становится корнем
- **кит** — d(кот, кит) = 1, метка свободна → ребро 1
- **ком** — d(кот, ком) = 1, занято китом → вниз в КИТ; d(кит, ком) = 2 → ребро 2 под КИТ
- **дом** — d(кот, дом) = 2, свободна → ребро 2
- **мир** — d(кот, мир) = 3, свободна → ребро 3

Форма дерева зависит от порядка вставки: те же слова в другом порядке дадут другое дерево, возможно сильно вытянутое.

BK-дерево: как ищем

Ищем «кос» с порогом k = 1: в узле считаем d(кос, узел) и спускаемся только в тех потомков, у которых метка ребра лежит в диапазоне [d − k, d + k].



Заметь: КИТ сам не подошёл (d = 2), но привёл к КОМ. Неподходящий узел всё равно может вести к ответам.

Ход поиска

1. d(кос, КОТ) = 1 ≤ 1 → КОТ в ответ
2. Диапазон [1−1, 1+1] = [0, 2]
3. КИТ и ДОМ — заходим
4. МИР (ребро 3) — отброшен целиком
5. Под КИТ диапазон [1, 3] → КОМ, d = 1 ✓

Почему корректно

Неравенство треугольника: за пределами [d−k, d+k] подходящих слов быть не может.
В каждом посещённом узле расстояние считается заново — переиспользовать нечего.

Три структуры: что выбрать

Все три решают одну задачу — найти слова в пределах порога k. Отличаются ценой.

Trie с отсечкой

- Дерево префиксов, по строке матрицы на узел
- Память: компактно, порядок слов сохранён
- Скорость: хорошая, обход в глубину

Когда нужны ещё и префиксные подсказки

SymSpell

- Хэш-таблица удалений, поиск = лукапы
- Память: индекс раздувается в десятки раз
- Скорость: самая высокая, без обхода

Когда память есть, а латентность критична

BK-дерево

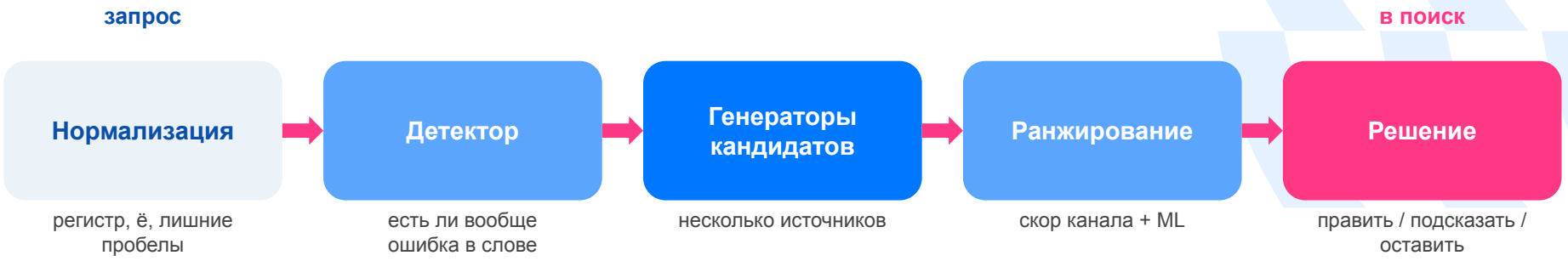
- Метрическое дерево, рёбра — расстояния
- Память: самое компактное из трёх
- Скорость: много случайных обращений

Когда словарь большой, а памяти мало

В проде часто держат две: быструю на основной поток и точную на остаток

Продакшн-опечаточник

В проде это не один алгоритм, а пайплайн. Большая часть кода — про выбор между гипотезами.



Whitelist

Бренды и названия, которые нельзя править ни при каких условиях.

Проверяется до детектора и обрывает пайплайн.

Источники кандидатов

- Словарный: trie / SymSpell
- Раскладка: ghbdtn → привет
- Транслит: privet → привет
- Пробелы: склейка и разбиение

Решение на выходе

высокая уверенность

исправляем молча

средняя

правим и показываем «искали X?»

низкая

оставляем как есть

Цена ошибки несимметрична: не исправили — пустая выдача, исправили неверно — чужая выдача и уход

Детектор: надо ли вообще править

Большинство слов набраны верно. Гонять генерацию кандидатов на каждое слово дорого и опасно.

Наивно: нет в словаре → ошибка

Ломается сразу с двух сторон:

- ✕ Реальнословные ошибки проходят насквозь: «кот» вместо «код» — оба слова в словаре
- ✕ Новое имя собственное помечается ошибкой и правится в чужое слово

Цена ошибки детектора

Пропустили опечатку — пользователь увидит пустую выдачу.
Сработали зря — испортили правильный запрос.

Что используют на самом деле

Частота	слово в словаре есть, но встретилось 3 раза, а рядом частотный сосед на расстоянии 1
Контекст	n-граммная модель оценивает, вероятно ли слово рядом с соседями по запросу
Классификатор	бинарная модель на этих признаках. В JamSpell — отдельный CatBoost «есть ли ошибка»
Поведение	ничего не кликнули и тут же переформулировали — сильный сигнал, что была опечатка

Поэтому порог детектора настраивают асимметрично, а не по максимуму accuracy

Ранжирование кандидатов

На входе десятки кандидатов из разных генераторов. Надо выбрать один и оценить уверенность.

База: скор зашумлённого канала

$$\log P(q \mid c) + \lambda \cdot \log P(c)$$

взвешенное расстояние плюс языковая модель — то, что разбирали в четвёртом разделе

Модель и выход

1. Бустинг на признаках: CatBoost, LightGBM
2. В JamSpell это регрессор CatBoost
3. Выход — скор, берём топ-1
4. Он же сравнивается с порогами решения

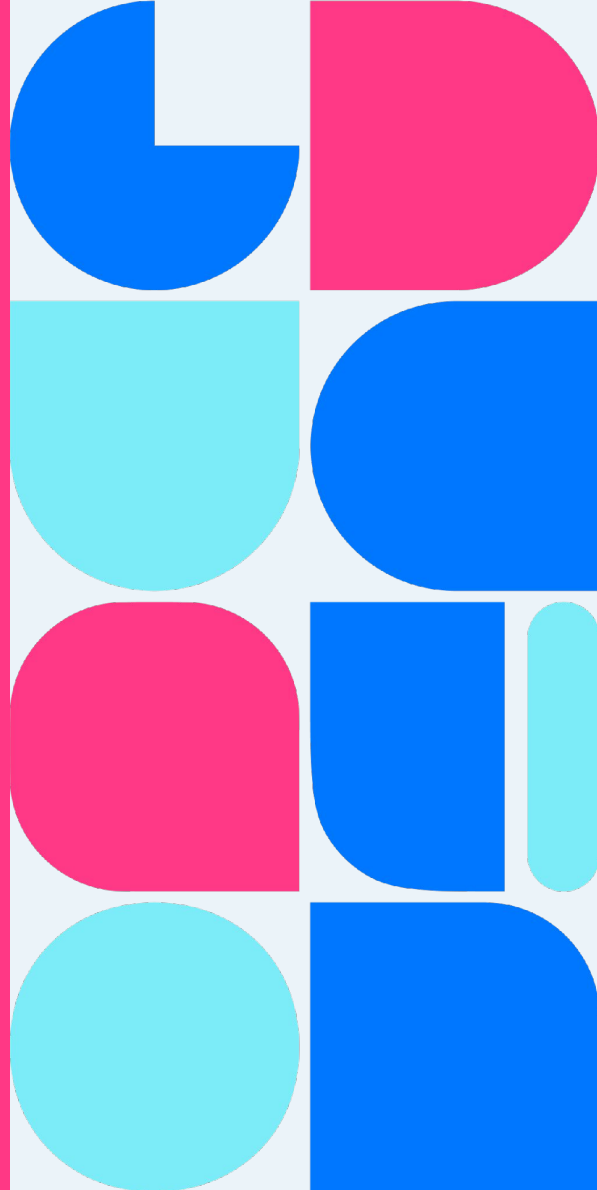
Признаки поверх базы

- Взвешенное расстояние Дамерау-Левенштейна
- Частота кандидата в логах и в корпусе
- Частоты биграмм с соседними словами запроса
- Предсказание n-граммной языковой модели
- Длина слова и длина общего префикса
- Из какого генератора пришёл кандидат
- Клики: как часто по этому исправлению переходили

Ровно та же схема, что в пирамиде ранжирования из первой лекции: дешёвый отбор, затем дорогая модель

Современные методы

Разобраться, что умеют нейросетевые исправители и когда они оправданы



Где классика упирается в потолок

- Контекст ограничен биграммой: модель видит одно соседнее слово, а не смысл запроса целиком
- Словарь нужно поддерживать: любое новое имя собственное требует ручной или полуручной работы
- Реальнословные ошибки не ловятся вовсе: «кот» вместо «код» — оба слова есть в словаре, расстояние 1, никакого сигнала
- Ошибки с большим расстоянием правки проходят мимо порога k
- Нейросетевые подходы решают это иначе: не «слово → ближайшее слово», а «испорченный текст → правильный текст»

Ключевая смена постановки

Классика: найти кандидатов и выбрать лучшего.
Нейросети: сгенерировать правильную строку целиком. Из второй постановки словарь и расстояние просто исчезают.

NeuSpell

- Открытый тулкит для контекстного исправления, EMNLP 2020 (Jayanthi, Pruthi, Neubig)
- Десять моделей в одном интерфейсе: SC-LSTM, CHAR-CNN-LSTM, BERT, комбинации с ELMo
- Постановка — sequence labeling: для каждого токена модель предсказывает распределение по словарю, то есть либо оставить слово, либо заменить
- Обучающие данные синтетические: авторы портят правильные тексты — меняют, переставляют и удаляют символы, подставляют частые неверные написания
- На BEA60K модель на BERT даёт F1 около 72

Осторожно при использовании

Тулкит только для английского. Последнее содержательное обновление — апрель 2021, репозиторий не трогали с июля 2023. Для русского языка брать его напрямую нельзя: нужен свой претрейн и свой словарь.

SAGE: постановка

- Открытый проект ai-forever (SberDevices / AIRI). Название расшифровывается как Spell checking via Augmentation and Generative distribution Emulation
- Статья: Martynov et al., принята на EACL 2024 (Findings). Текущий релиз библиотеки — v1.1.0, апрель 2024
- Постановка seq2seq: на вход подаётся предложение с ошибками, на выходе модель генерирует исправленное предложение целиком
- Никакого словаря и никакого расстояния — только энкодер-декодер
- Главный вклад работы не в архитектуре, а в том, как генерируются данные для обучения

Почему данные важнее архитектуры

Размеченных пар «ошибка → исправление» мало. Значит, нужно портить чистые тексты синтетически. Но случайный шум не похож на то, как ошибаются живые люди, и модель, обученная на нём, в проде не работает.

SAGE: генерация данных (SBSC)

SBSC — **Statistic-Based Spelling Corruption**. Метод «подсматривает» статистику реальных ошибок и воспроизводит её на чистом тексте.

Шаг 1: анализ

Берём параллельный корпус «ошибка → исправление».

Для каждой пары строим матрицу Левенштейна и идём backtrace — ровно то, что мы разбирали в третьем разделе.

Получаем: тип ошибки, её абсолютную и относительную позицию в предложении, число ошибок на предложение.

Шаг 2: воспроизведение

Все собранные статистики нормируем в дискретные распределения.

Для нового чистого предложения сэмплируем из них число ошибок, их типы и позиции — и применяем.

Получается синтетика, распределение ошибок в которой совпадает с настоящим.

Второй метод в SAGE — **Augmentex**: правила и эвристики поверх статистики KartaSlov — промахи по соседним клавишам, частые неверные написания, перестановки. Применяют оба.

SAGE: как обучают

Две стадии, и главный выигрыш даёт именно первая.

Предобучение

1. Берём большой корпус чистых текстов: русская Википедия и расшифровки видео
2. Портим его с помощью SBSC и Augmentex
3. Учим seq2seq: на вход испорченное предложение, на выходе корректное

Дообучение

Файнтюн на датасетах с человеческими ошибками: RUSpellRU, MultidomainGold и др. Именно связка «претрейн на синтетике + файнтюн на живых ошибках» даёт лучший результат.

Актуальные чекпоинты: sage-fredt5-large и дистиллированный sage-fredt5-distilled-95m для русского, sage-mt5-large и T5-large-spell для английского.

SAGE: бенчмарки

RUSpellRU, орфографические ошибки. Precision / Recall / F1 в процентах.

Модель	Precision	Recall	F1
sage-ai-service (сервис)	90.3	86.3	**88.2**
sage-fredt5-large (ft.)	88.4	80.9	**84.5**
sage-fredt5-distilled-95m (ft.)	83.5	74.8	**78.9**
gpt-4	54.9	76.7	**64.0**
sage-fredt5-large (zero-shot)	57.3	68.0	**62.2**
gpt-3.5-turbo	33.6	58.5	**42.7**

Что тут интересно

Дообученная модель на 95 млн параметров обходит GPT-4 по F1 почти на 15 пунктов. Универсальная LLM плохо решает задачу редактирования: у неё высокий recall, но она правит лишнее, и precision проваливается.

JamSpell: классический ML

- Библиотека на C++ (bakwc), обёртки для Python и других языков. Претрейны для 13 языков, включая русский
- Генерация кандидатов — модифицированный SymSpell
- Ранжирование — словная триграммная языковая модель плюс две модели CatBoost
- Первая модель, бинарный классификатор: есть ли вообще ошибка в этом слове
- Вторая, регрессор: какой из кандидатов лучший
- Признаки: частота слова, наличие в словаре, длина, частоты n-грамм, частоты соседних слов, предсказание n-граммной модели, редакционное расстояние
- Память экономят фильтром Блума и идеальным хэшированием; скорость — порядка тысяч слов в секунду

По сути это ровно наша схема из лекции: кандидаты плюс языковая модель — но финальное решение принимает не формула, а бустинг на табличных признаках. Хороший ориентир для домашнего задания.

Что выбрать под задачу

Классика на словаре

- \+ Десятки микросекунд
- \+ Полный контроль и объяснимость
- \+ Легко чинить точечно
- – Нужен и словарь, и статистика
- – Не ловит реальнословные ошибки

JamSpell и похожие

- \+ Контекст через триграммы
- \+ Готовые претрейны
- \+ Работает на CPU
- – Качество упирается в n-граммы
- – Обновление словаря — всё ещё ваша забота

SAGE и seq2seq

- \+ Лучшее качество на русском
- \+ Словарь не нужен вовсе
- \+ Чинит пунктуацию и регистр
- – Нужен GPU или дистилляция
- – Может «переписать» запрос

В проде почти всегда гибрид: быстрая классика на большинстве запросов и тяжёлая модель на тех, где она не уверена

А что LLM

- Соблазнительно: отдать запрос большой модели и попросить исправить. На практике это плохой дефолт
- Дорого и медленно: бюджет на предобработку — десятки миллисекунд, а не секунды
- Цифры со слайда бенчмарков: GPT-4 на RUSpellRU проигрывает дообученной 95-миллионной модели
- Универсальная модель склонна переписывать запрос, а не править опечатку: recall высокий, precision низкий
- Где LLM реально полезны: разметка обучающих данных, генерация правдоподобных ошибок, разбор сложных и редких случаев офлайн
- Общее правило: специализированная маленькая модель на своей задаче обходит универсальную большую

Это тот же вывод, что и в первой лекции про генеративный слой: LLM не заменяет компоненты поиска, а надстраивается над ними там, где может себе позволить время на размышление.

Итоги

- ✓ Опечатки есть в 10–20 % запросов, и в поиске они устроены иначе, чем в тексте
- ✓ Простой опечаточник = словарь + ближайшее слово. Ломается на трёх вещах: много равных кандидатов, сложный словарь, нет контекста
- ✓ Расстояние Левенштейна и алгоритм Вагнера — Фишера дают и число, и список конкретных операций
- ✓ Модель зашумлённого канала разносит задачу на модель ошибок и языковую модель
- ✓ Генерация кандидатов — отдельная задача: trie с отсечкой, SymSpell, BK-дерево
- ✓ Современный подход — seq2seq без словаря, и решает там не архитектура, а качество синтетики

Материалы

Почитать

- Jurafsky, Martin. Speech and Language Processing — глава про spelling correction и n-граммы
- P. Norvig. How to Write a Spelling Corrector — 20 строк, с которых стоит начать
- Martynov et al. A Methodology for Generative Spelling Correction, EACL 2024 Findings
- Jayanthi et al. NeuSpell, EMNLP 2020

Потрогать руками

- github.com/ai-forever/sage — модели, SBSC, бенчмарки
- github.com/bakwc/JamSpell — C++ и Python, претрейны для 13 языков
- github.com/wolfgarbe/SymSpell — референсная реализация
- kartaslov.ru — открытые данные по русским опечаткам
- hunspell.github.io — классический словарный спеллчекер

Спасибо
за внимание!

Руслан Аллаяров

education